



PostgreSQL

Up and Running

04

8

23 494 .Y[®]

Regina Obe & Leo Hsu

5. You don't need to run the slaves on the same port as the master, so you can optionally change the port either via *postgresql.conf* or via some other OS specific startup script that sets `PGPORT` before startup. Any startup script will override the setting you have in *postgresql.conf*.
6. Create a new file in the *data* folder called *recovery.conf* that contains the following lines:

```
standby_mode = 'on'  
primary_conninfo = 'host=192.168.0.1 port=5432 user=pgrepuser password=woohoo'  
trigger_file = 'failover.now'
```

Host name, IP, and port should be those of the master.

7. Add to the *recovery.conf* file the following line, which varies, depending on the OS:
On Linux/Unix:

```
restore_command = 'cp %p ../archive/%f'
```

On Windows:

```
restore_command = 'copy %p ..\\archive\\%f'
```

This command is only needed if the slave can't play the WALs fast enough, so it needs a location to cache them.

Initiate the Replication Process

1. Start up the slave server first. You'll get an error in logs that it can't connect to the master. Ignore.
2. Start up the master server.

You should now be able to connect to both servers. Any changes you make on the master, even structural changes like installing extensions or creating tables, should trickle down to the slaves. You should also be able to query the slaves, but not much else.

When and if the time comes to liberate a chosen slave, create a blank file called *failover.now* in the *data* folder of the slave. What happens next is that Postgres will complete the playing back of WAL, rename the *recover.conf* to *recover.done*. At that point, your slave will be unshackled from the master and continue life on its own with all the data from the last WAL. Once the slave has tasted freedom, there's no going back. In order to make it a slave again, you'll need to go through the whole process from the beginning.



Unlogged tables don't participate in replication.

Foreign Data Wrappers (FDW)

Foreign Data Wrappers are mechanisms of querying external datasources. PostgreSQL 9.1 introduced this SQL/MED standards compliant feature. At the center of the concept is what is called a foreign table. In this section, we'll demonstrate how to register foreign servers, foreign users, and foreign tables, and finally, how to query foreign tables. You can find a catalog of foreign data wrappers for PostgreSQL at PGXN FDW (<http://pgxn.org/tag/fdw/>) and PGXN Foreign Data Wrapper (<http://pgxn.org/tag/foreign%20data%20wrapper/>). You can also find examples of usage in PostgreSQL Wiki FDW (http://wiki.postgresql.org/wiki/Foreign_data_wrappers). At this time, it's rare to find FDWs packaged with PostgreSQL except for *fdw_file* (<http://www.postgresql.org/docs/current/interactive/file-fdw.html>). For wrapping anything else, you'll need to compile your own or get them from someone who already did the work. In PostgreSQL 9.3, you can expect a FDW that will at least wrap other PostgreSQL databases. Also, you're limited to SELECT queries against the FDW, but this will hopefully change in the future so that you can use them to update foreign data as well.

Querying Simple Flat File Data Sources

We'll gain an introduction to FDW using the *file_fdw* wrapper. To install, use the command:

```
CREATE EXTENSION file_fdw;
```

Although *file_fdw* can only read from files on your local server, you still need to define a server for it. You register a FDW server with the following command.

```
CREATE SERVER my_server FOREIGN DATA WRAPPER file_fdw;
```

Next, you have to register the tables. You can place foreign tables in any schema you want. We usually create a separate schema to house foreign data. For this example, we'll use our staging schema.

Example 10-1. Make a Foreign Table from Delimited file

```
CREATE FOREIGN TABLE staging.devs (developer VARCHAR(150), company VARCHAR(150))  
SERVER my_server  
OPTIONS (format 'csv', header 'true', filename '/postgresql_book/ch10/devs.psv', delimiter  
'|', null '');
```

When all the set up is finished, we can finally query our pipe delimited file directly:

```
SELECT * FROM staging.devs WHERE developer LIKE 'T%';
```

Once we no longer need our foreign table, we can drop it with the basic SQL command:

```
DROP FOREIGN TABLE staging.devs;
```

Querying More Complex Data Sources

The database world does not appear to be getting more homogeneous. We're witnessing exotic databases sprouting up left and right. Some are fads that go away. Some aspire to dethrone the relational databases altogether. Some could hardly be considered databases. The introduction of foreign data wrappers is in part a response to the growing diversity. Resistance is futile. FDW assimilates.

In this next example, we'll demonstrate how to use the `www_fdw` foreign data wrapper to query web services. We borrowed the example from *www_fdw Examples* (https://github.com/cyga/www_fdw/wiki/Examples).



The `www_fdw` foreign data wrapper is not generally packaged with PostgreSQL installs. If you are on Linux/Unix, it's an easy compile if you have the `postgresql-dev` installed. We did the work of compiling for Windows—you can download our binaries here: *Windows-32 9.1 FDWs* (http://www.postgresqlonline.com/downloads/fdw_win32_91_bin.zip).

The first step to perform after you have copied the binaries and extension files is to install the extension in your database:

```
CREATE EXTENSION www_fdw;
```

We then create our Twitter foreign data server:

```
CREATE SERVER www_fdw_server_twitter
FOREIGN DATA WRAPPER www_fdw
OPTIONS (uri 'http://search.twitter.com/search.json');
```

The default format supported by the `www_fdw` is JSON, so we didn't need to include it in the `OPTIONS` modifier. The other supported format is XML. For details on additional parameters that you can set, refer to the *www_fdw documentation* (https://github.com/cyga/www_fdw/wiki/Documentation). Each FDW is different and comes with its own API settings.

Next, we define at least one user for our FDW. All users that connect to our server should be able to access the Twitter server, so we create one for the entire public group.

```
CREATE USER MAPPING FOR public SERVER www_fdw_server_twitter;
```

Now we create our foreign table:

Example 10-2. Make a Foreign Table from Twitter

```
CREATE FOREIGN TABLE www_fdw_twitter (
/* parameters used in request */
q text,    page text, rpp text, result_type text,
/* fields in response */
created_at text, from_user text, from_user_id text,    from_user_id_str text
, geo text, id text, id_str text
```

```
, is_language_code text, profile_image_url text
, source text, text text, to_user text, to_user_id text)
SERVER www_fdw_server_twitter;
```

The user mapping doesn't imply rights. We still need to grants rights before being able to query the foreign table.

```
GRANT SELECT ON TABLE www_fdw_twitter TO public;
```

Now comes the fun part. Here, we ask for page two of any tweets that have something to do with postgresql, mysql, and nosql:

```
SELECT DISTINCT left(text,75) As part_txt
FROM www_fdw_twitter WHERE q='postgresql AND mysql AND nosql' and
page='2';
```

Voilà! We have our response:

```

                                part_txt
-----
MySQL Is Done. NoSQL Is Done. It's the Postgres Age http://t.co/4DfqG75d
RT @mjasay: .@451Research: <0.002% of paid MySQL deployments being repla
@alanzeino: I know MySQL... but anyone with a brain is using PostgreSQL
Hstore FTW! RT @mjasay: .@451Research: <0.002% of MySQL deployments bein
@al3xandru: MySQL Is Done. NoSQL Is Done. It's the Postgres Age http://t
```